DISEÑO DE SOFTWARE

Diseño en el Nivel de Componentes

Visiones, Principios y Realización

Repaso — Las tres visiones de un componente

Clase anterior: Conceptos Fundamentales | Hoy: profundizamos en cada visión

01

Visión Orientada a Objetos

El componente es un conjunto de clases colaborativas. Cada clase tiene atributos, métodos e interfaces bien definidas. (HOY: profundizamos en esto)

02

Visión Tradicional (funcional)

El componente agrupa funciones y módulos por lógica común. Se diseña pensando en transformaciones de datos y flujo del programa.

03

Visión del Proceso (reutilizable)

El componente se selecciona de una librería existente, se adapta al sistema y se integra. Máxima reutilización.

LECCIÓN 2

Visión Orientada a Objetos

Clases, relaciones e interfaces como base del diseño

¿Qué es el diseño OO de componentes?

Diseñar significa definir CÓMO las clases se estructuran y relacionan

En la visión OO, un componente = conjunto de clases colaborativas. Diseñar significa decidir CÓMO esas clases se estructuran, relacionan y comunican.

1. Identificar clases

¿Qué entidades del problema se convierten en clases?
Ej: Alumno, Curso, Nota, Profesor, Matrícula

2. Definir relaciones

¿Cómo se relacionan entre sí?
Herencia (es-un), composición (tiene-un), asociación, dependencia

3. Especificar interfaces

¿Qué métodos y atributos expone cada clase al exterior?
Visibilidad (+público, -privado, #protegido) y contratos claros

yo: clases con ALTA COHESIÓN, BAJO ACOPLAMIENTO e interfaces bien definidas.

Diagramas de clase UML — Herramienta del diseño OO

Representamos el diseño antes de escribir código

El diagrama de clase UML es el lenguaje universal para representar el diseño OO. Muestra clases, atributos, métodos y relaciones.

Alumno

```
- nombre: str
- notas: list
- numero: int
```

```
+ agregar_nota(n)
+ promedio(): float
+ get_nombre(): str
```

Curso

```
- nombre: str
- codigo: str
- alumnos: list
```

```
+ inscribir(alumno)
+ listar_alumnos()
+ promedio_curso()
```

Profesor

```
- nombre: str
- especialidad: str
```

```
+ asignar_curso(c)
+ registrar_nota(a,n)
+ ver_alumnos()
```

Relaciones UML: — Asociación (usa/conoce) ◆ Composición (contiene/parte de) △ Herencia (es un tipo de) ··· Dependencia

🎯 *Antes de escribir código: dibuja el diagrama. Define qué clases existen y cómo se relacionan.*

LECCIÓN 2 — CONT.

Visiones Tradicional y del Proceso

Otras formas de pensar los componentes

Visión Tradicional — Componentes funcionales

El componente agrupa funciones relacionadas por lógica, no por objetos

En la visión tradicional (funcional), un componente = módulo con funciones y procedimientos agrupados por su lógica de negocio.

Módulo funcional

Agrupa funciones que trabajan sobre los mismos datos o realizan tareas de la misma área. Ej: módulo_notas, módulo_alumnos.

Transformación de datos

El diseño se piensa como flujo: entrada → proceso → salida. Cada función transforma los datos y los pasa al siguiente módulo.

Interfaz de módulo

Cada módulo define qué funciones expone al exterior (público) y cuáles son internas. Similar a una API interna del programa.

Aplicación práctica

Útil en sistemas legacy (C, Pascal, scripts), procesamiento por lotes, herramientas CLI y programas sin estado complejo.

Visión del Proceso — Reutilización de componentes

Seleccionar, adaptar e integrar: aprovechar lo que ya existe

En la visión del proceso, los componentes se seleccionan de librerías existentes y se reutilizan. El diseño es sobre integración, no creación desde cero.

1

Identificar el componente

Buscar en librerías internas o externas un componente que resuelva el problema. Ej: una librería de cálculo estadístico para promedios.

2

Calificar e inspeccionar

Verificar que el componente cumpla con los requisitos: funcionalidad, rendimiento, compatibilidad, licencia y calidad del código.

3

Adaptar e integrar

Si es necesario, adaptar la interfaz del componente al sistema (patrón Adapter). Luego integrarlo y validar que funcione con el resto.

♻ *Reutilizar > reinventar. Un buen componente creado hoy puede usarse en 5 proyectos futuros.*

LECCIÓN 2 — CONT.

Diseño de Componentes Basados en Clases

Principios SOLID y lineamientos prácticos

Principios de diseño — SOLID

5 principios para crear clases bien diseñadas, mantenibles y extensibles

S	Single Responsibility	Una clase = UN solo motivo para cambiar. Si mezcla datos, lógica y persistencia → dividir en 3 clases.
O	Open / Closed	Abierto para extender, cerrado para modificar. Agregar funcionalidad creando clases nuevas, no editando las existentes.
L	Liskov Substitution	Una subclase debe poder reemplazar al padre sin cambiar el comportamiento. Si la subclase rompe el contrato → no heredes.
I	Interface Segregation	Muchas interfaces específicas > una interfaz general. No obligar a implementar métodos que no se usan.
D	Dependency Inversion	Depender de abstracciones, no de implementaciones concretas. Usa interfaces entre capas del sistema.

S

Single Responsibility — Responsabilidad Única

Una clase, un solo motivo para cambiar

¿Qué significa?

Cada clase debe encargarse de una sola cosa. Si una clase maneja datos del alumno, guarda en la base de datos Y envía correos, tiene demasiadas responsabilidades.

Idea clave

Si necesitas cambiar cómo se envían los correos, no deberías tener que tocar la clase Alumno. Cada cambio debe afectar solo a la clase que le corresponde.

💡 *Analogía: un chef cocina, un mesero sirve, un cajero cobra. Cada uno tiene su rol. Si el chef también cobra y sirve, todo se desordena.*

O

Open / Closed — Abierto/Cerrado

Abierto para extender, cerrado para modificar

¿Qué significa?

Debes poder agregar nueva funcionalidad al sistema sin tener que modificar el código que ya existe y funciona correctamente.

Idea clave

Cada vez que editas código existente para agregar algo nuevo, arriesgas romper lo que ya funcionaba. La solución: crear cosas nuevas en lugar de editar las viejas.

💡 *Analogía: cuando compras un accesorio nuevo para tu teléfono no abres el teléfono y lo modificas por dentro — simplemente lo conectas.*

L

Liskov Substitution — Sustitución de Liskov

Una subclase debe poder reemplazar al padre sin romper el programa

¿Qué significa?

Si tienes una clase Animal y una subclase Perro, deberías poder usar un Perro en cualquier lugar donde se usa un Animal sin que el programa falle o se comporte de forma inesperada.

Idea clave

Una subclase que cambia o elimina el comportamiento del padre viola este principio. Heredar solo tiene sentido si la subclase es realmente "un tipo de" la clase padre en todos los aspectos.

💡 *Analogía: si contratas a un suplente para un trabajo, esperas que haga el mismo trabajo que el titular. Si hace cosas completamente distintas, no es un buen suplente.*

I

Interface Segregation — Segregación de Interfaces

No obligues a una clase a implementar lo que no usa

¿Qué significa?

Es mejor tener varias interfaces pequeñas y específicas que una sola interfaz grande y general. Cada clase implementa solo lo que realmente necesita.

Idea clave

Si una impresora simple se ve obligada a implementar también 'escanear' y 'enviar fax' solo porque la interfaz Maquina los incluye, el diseño está mal. La clase carga con responsabilidades ajenas.

💡 *Analogía: no le das a un empleado nuevo un contrato con 40 cláusulas si solo necesita hacer una tarea. Le das un contrato simple y específico.*

D

Dependency Inversion — Inversión de Dependencias

Depende de abstracciones, no de clases concretas

¿Qué significa?

Los módulos de alto nivel no deben depender directamente de los módulos de bajo nivel. Ambos deben depender de una abstracción (interfaz o clase base) que los desacople.

Idea clave

Si la clase Notificador crea directamente un ServicioCorreo, está atada al correo para siempre. Si en cambio recibe un EnviadorMensaje genérico, puedes cambiarlo por SMS o WhatsApp sin tocar Notificador.

💡 *Analogía: el enchufe de la pared no depende de qué aparato conectes. Define un contrato (la forma del enchufe) y cualquier aparato que lo cumpla puede usarlo.*

LECCIÓN 3

Lineamientos de Diseño en el Nivel de Componentes

Reglas prácticas para tomar mejores decisiones de diseño

11° Año | Diseño de Software

Lineamientos de diseño — Nivel de componentes

Reglas prácticas que guían las decisiones de diseño del día a día

01

Nombra las clases con sustantivos

Las clases representan entidades: Alumno, Factura, Pedido. Los métodos usan verbos: calcular(), guardar(), validar().

02

Define la interfaz antes de implementar

Decide qué servicios ofrece la clase (métodos públicos) antes de escribir el código interno. El contrato primero.

03

Minimiza la colaboración — Ley de Demeter

"No hables con extraños": un método solo debe llamar a su propia clase, sus parámetros, sus atributos y objetos que crea.

04

Prefiere composición sobre herencia

En lugar de heredar comportamiento, incluye objetos dentro. Más flexible, menos frágil. Sigue el principio SOLID O.

05

Mantén las clases pequeñas y enfocadas

Si una clase ocupa más de una pantalla, probablemente está haciendo demasiado. Divide y vencerás.

Ley de Demeter — "No hables con extraños"

Un componente solo debe comunicarse con sus vecinos directos, no con desconocidos

¿Qué significa?

Un método solo debería interactuar con su propia clase, con los objetos que recibe como parámetros, con sus atributos directos y con objetos que él mismo crea. Nada más.

Idea clave

Cuando un componente conoce demasiados detalles internos de otros, se vuelve frágil: cualquier cambio en esos otros lo rompe. Cuanto menos sabe un componente de sus vecinos, más fácil es cambiar el sistema.

💡 *Analogía: si quieres saber la hora, miras tu reloj directamente. No le preguntas a tu amigo que le pregunta a su jefe que consulta al servidor. Menos saltos = menos dependencias.*

Herencia — "es un tipo de"

Una clase hija reutiliza y extiende el comportamiento de una clase padre

¿Qué significa?

Cuando una clase hereda de otra, recibe automáticamente todos sus atributos y métodos. La clase hija puede usarlos tal cual o modificarlos para adaptarlos a su caso particular.

Idea clave

Úsala cuando la relación es real y permanente: un Perro siempre será un Animal, un Estudiante siempre será una Persona. Evítala cuando la relación es forzada o puede cambiar con el tiempo, porque un cambio en la clase padre afecta a todas las clases hijas.

💡 *Ejemplo: Animal tiene el método respirar(). Perro hereda de Animal y además agrega ladrar(). No hay que escribir respirar() de nuevo — ya lo tiene por herencia.*

Composición — "tiene un"

Una clase se construye incluyendo otras clases como partes de sí misma

¿Qué significa?

En lugar de heredar de otra clase, una clase contiene objetos de otras clases dentro de sí. Esos objetos son sus "piezas". La clase principal los usa para hacer su trabajo, pero cada pieza puede cambiarse de forma independiente.

Idea clave

Es más flexible que la herencia porque puedes cambiar o reemplazar una pieza sin tocar el resto. Si un Auto cambia su Motor por uno eléctrico, solo cambias esa pieza. El resto del auto sigue igual. Por eso el diseño moderno la prefiere.

💡 *Ejemplo: un Auto no hereda de Motor — tiene un Motor. Si mañana quieres cambiar el motor de gasolina por uno eléctrico, solo cambias esa pieza. El resto del auto no se toca.*

LECCIÓN 4

Realización del Diseño en el Nivel de Componentes

Del diagrama UML al código real: el proceso completo

11° Año | Diseño de Software

El proceso de realización — Del diseño al código

Realizar el diseño = tomar las decisiones de diseño y convertirlas en código ejecutable

1

Identificar clases del diseño

A partir del análisis del problema, determinar qué entidades del mundo real se convierten en clases del sistema.

2

Especificar los atributos

Definir qué datos guarda cada clase: nombre, tipo y visibilidad (+público, -privado, #protegido).

3

Especificar las operaciones

Definir qué puede hacer cada clase: nombre del método, parámetros que recibe y qué devuelve.

4

Definir las interfaces

Establecer el contrato de cada componente: qué ofrece al exterior. Solo la interfaz, nunca la implementación interna.

5

Elaborar pseudocódigo

Para operaciones complejas, describir la lógica en lenguaje natural antes de escribir código. Ayuda a detectar errores de diseño temprano.

6

Implementar y validar

Traducir el diseño a código real y verificar que cumple con los requisitos y principios establecidos.

De UML a la implementación — El puente entre diseño y código

El diagrama UML no es un adorno — es el plano que guía cada línea de código

¿Qué significa?

Cada elemento del diagrama de clase UML tiene una traducción directa al código. El nombre de la clase, sus atributos con tipos y visibilidad, y sus operaciones con parámetros y retorno se convierten en la estructura real del programa.

Idea clave

Si el diseño UML está bien hecho — clases bien nombradas, atributos claros, operaciones definidas — implementar en código es casi mecánico. El esfuerzo intelectual real ocurre en el diseño, no en la escritura.

💡 *Analogía: el arquitecto dibuja el plano del edificio antes de construir. El plano define cada cuarto, cada pared, cada puerta. Los obreros construyen siguiendo ese plano. El diagrama UML es el plano del software.*

Pseudocódigo — Piensa antes de escribir

Describir la lógica en lenguaje natural antes de comprometerse con un lenguaje de programación

¿Qué significa?

El pseudocódigo es una descripción de la lógica de un algoritmo usando lenguaje natural estructurado. No sigue la sintaxis de ningún lenguaje de programación específico — su único propósito es pensar con claridad antes de escribir código real.

Idea clave

Es mucho más fácil detectar un error lógico en un pseudocódigo de 10 líneas que encontrarlo después de haber escrito 100 líneas de código real. El pseudocódigo obliga a pensar en todos los casos posibles: ¿qué pasa si el dato es inválido? ¿Y si está vacío?

💡 *Analogía: antes de construir una casa, haces un borrador en papel. Antes de cocinar un plato nuevo, mentalmente repasas los pasos. El pseudocódigo es ese borrador mental hecho visible.*

Lo que aprendimos hoy

3 Visiones: OO (clases y relaciones), Tradicional (módulos y funciones), Proceso (librerías reutilizables).

SOLID: S=responsabilidad única, O=abierto/cerrado, L=Liskov, I=segregación, D=inversión de dependencia.

Lineamientos: Sustantivos para clases, verbos para métodos, Ley de Demeter, composición > herencia.

Realización: 6 pasos del diseño al código: identificar, atributos, operaciones, interfaces, pseudocódigo, implementar.

UML → Código: El diagrama de clase se convierte directamente en código. Atributos y operaciones tienen visibilidad clara.

Próxima clase: Patrones de diseño aplicados a componentes